

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 02 -

**GERENCIAMENTO DE PACOTES E
AMBIENTES VIRTUAIS (POETRY E
UV)**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

O gerenciamento de pacotes no Python passou por muitas melhorias nos últimos anos, com a comunidade percebendo que a solução com o `requirements.txt` não era suficiente para atender todas as necessidades que os diferentes tipos de projetos que são criados na linguagem possuem.

Duas ferramentas modernas se destacam entre as que foram criadas, o `uv` e o `poetry`, cada uma resolvendo os problemas do seu jeito, mas com o mesmo objetivo: garantir que as ferramentas e bibliotecas usadas pelo software criado usando Python, independentemente de suas características, funcione como o esperado.

HANDS ON

Todo o gerenciamento de dependências é feito com o uso de apenas um arquivo, `pyproject.toml`, e este arquivo contém também as configurações do projeto e outras informações que vamos conhecer posteriormente. Mas não vamos editar este arquivo manualmente, pelo menos na maior parte do tempo, porque sendo um arquivo mais complexo, a chance de erros aumenta muito.

Por isso, também vamos conhecer duas ferramentas usadas para fazer o gerenciamento de dependências e ambientes de operação, o moderno `uv` e o já estabelecido `poetry`.

SAIBA MAIS

O arquivo pyproject.toml

Antes de falar sobre as ferramentas, precisamos entender o coração das duas ferramentas: o arquivo `pyproject.toml`.

Antes de existir um padrão, a configuração de um projeto Python era feita em múltiplos arquivos, cada um com uma responsabilidade e funcionando de forma independente: o `setup.py` (ou `setup.cfg`) definia metadados e dependências para a instalação de um pacote, o `requirements.txt` listava pacotes para instalação de um ambiente, o `tox.ini` configurava testes, o `.flake8` configurava o linter e assim por diante. E cada nova ferramenta possivelmente teria seu próprio arquivo de configuração, o que tornava a raiz de qualquer projeto não trivial um cemitério de arquivos soltos sem relação entre si.

O `pyproject.toml` surgiu com a proposta de ser o ponto único de verdade para a configuração do projeto, independentemente de quais ferramentas fossem utilizadas. Pela definição, este arquivo foi criado para ser o local de configuração central para projetos Python, introduzido pela PEP 518 em 2016 e consolidado pelas PEPs 517 e 621.

Mas o problema que motivou sua criação vai além da organização de arquivos ou da facilidade de uso. Se usamos o `requirements.txt`, não conseguimos isolar quais bibliotecas tem que ser instaladas em qual ambiente, quais as dependências devem ser instaladas apenas no ambiente de desenvolvimento, quais as configurações necessárias para executar algum comando do processo.

Do ponto de vista da reprodutibilidade, que é o que estamos estudando neste momento, o `pyproject.toml` resolve três objetivos fundamentais: primeiro, centraliza e versiona toda a configuração do projeto em um único arquivo, que vive no repositório junto ao código; segundo, desacopla o projeto de uma ferramenta específica: ao definir um padrão aberto, qualquer build backend compatível (como `setuptools`, `poetry` ou `uv`) pode interpretar o arquivo, evitando dependências de arquivos personalizados por ferramenta; e terceiro, torna as configurações e ferramentas do projeto explícitas e auditáveis: dependências por ambiente, e com as versões

definidas, versão do Python requerida, configuração de testes, formatação e análise estática ficam documentadas de forma declarativa, o que é essencial para garantir que qualquer pessoa, em qualquer máquina, tenha as mesmas configurações de ambiente; e isso se replica também para a configuração de projetos rodando em ambientes de uso geral.

Mas na maioria das vezes não é necessário alterar diretamente o arquivo `pyproject.toml`; como citado anteriormente, a comunidade criou ferramentas que usam o arquivo como referência para gerenciar o ambiente de execução. Dentre estas ferramentas, destacaremos duas: o `uv` e o `poetry`.

O que é o uv

O `uv` é um gerenciador de pacotes e ambientes Python desenvolvido pela Astral, uma empresa que já desenvolveu outras ferramentas para Python, para diferentes usos. Ele foi projetado para ser um substituto unificado de várias ferramentas do ecossistema Python, como o `pip` para instalação de pacotes e o `virtualenv` para criação de ambientes virtuais. A proposta central é simples: uma única ferramenta, extremamente rápida, que cobre todo o ciclo de gerenciamento de um projeto Python.

E, nesse caso, a velocidade não é marketing: o `uv` é entre 10x e 100x mais rápido que o `pip` em operações de resolução e instalação de dependências. Para ciência de dados, onde ambientes costumam ter dezenas de pacotes pesados como `numpy`, `pandas`, `torch` e `scikit-learn`, essa diferença é sentida na prática.

Como funciona

O `uv` opera em duas responsabilidades principais, trabalhando em conjunto. Primeiro, o gerenciamento de runtimes Python. O runtime é o ambiente Python onde o código é executado, compreendendo o Python e suas bibliotecas padrão, e sempre identificado por um número de versão. Por exemplo, se sua aplicação depende do Python 3.12, na verdade dizemos que ela depende do runtime python, na versão 3.12. O `uv` pode instalar e gerenciar versões do próprio Python, sem depender de ferramentas externas ou até mesmo de instalações do sistema. Isso

significa que você pode declarar as versões do Python no seu projeto e o uv garante que essa versão esteja disponível, baixando-a e configurando se necessário.

O uv também cuida do gerenciamento de dependências e ambientes. Ele lê o `pyproject.toml` do projeto, resolve as dependências (ou seja, valida e instala as dependências) e cria ambientes virtuais isolados automaticamente. Ele também gera um arquivo `uv.lock` — um lockfile com as versões exatas e os hashes de todos os pacotes instalados, garantindo reprodutibilidade completa entre máquinas.

Internamente, o uv funciona assim: ao rodar um comando relacionado ao gerenciamento de dependências, ele lê as dependências declaradas, resolve o grafo de compatibilidade (que pode muitas vezes ser complexo, com dependências em diferentes níveis), consulta o cache local (evitando downloads desnecessários) e instala tudo no ambiente virtual do projeto, que fica em `.venv/` na raiz do projeto por padrão — sem mágica, sem diretórios globais escondidos e garantindo que o contexto do projeto seja o diretório atual.

Exemplos de uso

Para criar um projeto novo do zero, podemos utilizar:

```
uv init meu-projeto-ds
cd meu-projeto-ds
```

Isso gera a estrutura básica com `pyproject.toml` já configurado.

Para especificar a versão do Python, é possível utilizar: `uv python pin 3.11`

Ele cria um arquivo `.python-version` que o uv respeita automaticamente. Isso também baixa a versão do python desejada, caso não esteja disponível

Para adicionar dependências, basta digitar:

```
uv add pandas scikit-learn matplotlib
uv add --dev pytest ruff
```

O uv atualiza o `pyproject.toml`, resolve o ambiente e gera o `uv.lock`, tudo em uma única operação. Interessante notar que o primeiro comando instala as bibliotecas para o ambiente padrão e o segundo apenas para o ambiente dev, normalmente a nomenclatura para ambientes de desenvolvimento.

Para reproduzir o ambiente em outra máquina, basta utilizar:

```
uv sync
```

Com o `pyproject.toml` e o `uv.lock` versionados no repositório, esse único comando instala exatamente o mesmo ambiente, com as mesmas versões do Python e das bibliotecas. É aqui que a promessa de reprodutibilidade se mostra.

Para rodar scripts sem ativação manual do ambiente, utilizamos:

```
uv run python treino.py
```

```
uv run jupyter lab
```

```
uv run pytest
```

O `uv run` garante que o comando seja executado dentro do ambiente do projeto, sem precisar ativar o `.venv` explicitamente — útil em pipelines de CI e scripts automatizados e mesmo em desenvolvimento local. Muitas das IDEs e editores mais modernos já têm suporte nativo ao uv.

Já para instalar ferramentas globais isoladas, utilizamos:

```
uv tool install ruff
```

```
uv tool install jupyterlab
```

Ele instala ferramentas de linha de comando em ambientes isolados, sem poluir o ambiente global ou o do projeto. Isso é útil principalmente em ambientes locais, de desenvolvimento, onde precisamos de alguma ferramenta que não precisa estar instalada no contexto do projeto.

A combinação `pyproject.toml` + `uv.lock` + `uv sync` resolve um problema que o `pip` + `requirements.txt` nunca conseguiu atender completamente. O `requirements.txt` tradicional não distingue dependências diretas de transitivas, não garante versões

corretas de bibliotecas e não está integrado com o gerenciamento do runtime Python. O uv trata tudo isso com elegância, o que o torna uma escolha sólida tanto para projetos individuais de ciência de dados quanto para ambientes de produção em MLOps.

O que é o Poetry?

Outra ferramenta existente é o Poetry, focada em gerenciamento de dependências e empacotamento para Python. Assim como o uv, ele nasceu da insatisfação com o processo tedioso e complicado, propenso a erros, para fazer o que deveria ser simples: declarar dependências e garantir que o projeto funcione igualmente em qualquer lugar. O Poetry foi uma das primeiras ferramentas a oferecer essa experiência unificada de forma polida e por isso ganhou grande adoção na comunidade antes mesmo do uv existir.

O Poetry é escrito em Python e prioriza a experiência do indivíduo desenvolvedor em todo o processo. Ele tem uma interface de linha de comando bem construída, mensagens de retorno de comandos que ajudam a resolver os problemas de execução dos comandos executados e um fluxo de trabalho opinativo que guia o usuário em direção a boas práticas — o que o torna especialmente acessível para quem está estruturando projetos Python com mais seriedade pela primeira vez.

Como funciona

Como padrão, o Poetry também usa o `pyproject.toml`, utilizando uma seção própria `[tool.poetry]` para declarar metadados (dados do projeto, de mantenedores, versão do projeto etc), dependências e configurações do projeto. Assim como o uv, ele gera um lockfile — o `poetry.lock` — que registra as versões exatas e os hashes de todos os pacotes resolvidos, garantindo que qualquer pessoa que clone o repositório e rode o comando de sincronização obtenha exatamente o mesmo ambiente.

O gerenciamento de ambientes virtuais no Poetry também é automatizado: quando necessário, ele cria e gerencia um `.venv` associado ao projeto e todos os

comandos de execução passam por esse ambiente sem necessidade de ativação manual. Mas, diferente do uv, o Poetry não gerencia o runtime Python em si — ele depende de que a versão correta do interpretador já esteja instalada no sistema, seja via pyenv, instalador oficial (obtido no site) ou a existente por padrão no sistema operacional. A versão desejada é declarada no `pyproject.toml`, mas cabe a quem vai usar garantir que essa versão esteja disponível.

Exemplos de uso

Para criar um projeto novo do zero, basta utilizar:

```
poetry new meu-projeto-ds
cd meu-projeto-ds
```

Ele gera uma estrutura de projeto com `pyproject.toml`, diretório de código e estrutura básica de testes já configurada. É útil para iniciar um projeto do zero.

Já para iniciar um projeto existente, basta utilizar o:

```
poetry init
```

Este é o modo interativo que guia o usuário na criação do `pyproject.toml` com perguntas sobre nome, versão, dependências e versão do Python.

Para adicionar dependências, empregamos:

```
poetry add pandas scikit-learn matplotlib
poetry add --group dev pytest ruff
```

Assim como o `uv add`, o comando atualiza o `pyproject.toml` e o `poetry.lock` automaticamente, mantendo os dois sempre sincronizados.

Já para reproduzir o ambiente em outra máquina, temos:

```
poetry install
```

Com `pyproject.toml` e `poetry.lock` versionados no repositório, esse comando reconstrói o ambiente de forma determinística — mesmas versões, mesmos hashes, sem surpresas.

Para rodar comandos dentro do ambiente, temos:

```
poetry run python treino.py
```

```
poetry run jupyter lab
```

```
poetry run pytest
```

É equivalente ao `uv run`: ele executa o comando no contexto do ambiente do projeto sem precisar ativá-lo manualmente.

Por fim, para acessar o shell do ambiente, usamos:

```
poetry shell
```

Ele ativa o ambiente virtual de forma interativa no terminal — útil para sessões exploratórias, algo que o `uv` não oferece de forma equivalente por padrão. Para desativar o ambiente virtual, entre com o comando `deactivate`.

Poetry ou uv — qual usar?

Essa é uma pergunta interessante que deve surgir ao ver as duas ferramentas lado a lado, dado que as duas resolvem problemas muito parecidos. O Poetry é mais maduro, tem ecossistema de plugins, documentação extensa e uma base de usuários consolidada — é uma escolha segura para projetos que precisam de estabilidade e onde a experiência do indivíduo desenvolvedor é prioridade. O `uv`, por outro lado, é significativamente mais rápido, gerencia o próprio runtime Python e está convergindo rapidamente em termos de funcionalidades.

Para novos projetos, especialmente em contextos de MLOps e pipelines automatizados onde performance de CI importa, o `uv` tende a ser a escolha mais moderna. Para equipes já estabelecidas com Poetry, a migração nem sempre se justifica — as duas ferramentas entregam reprodutibilidade de forma confiável.

O QUE VOCÊ VIU NESTA AULA?

Partindo da premissa que entendemos que um dos principais desafios para criar ambiente reproduzíveis para os projetos é o gerenciamento de pacotes, vimos a solução moderna para solucionar os problemas de gerenciamento de ambientes e dependência, que é o uso do arquivo `pyproject.toml`, criado especificamente para endereçar essa deficiência do ferramental usado por quem cria e entrega soluções usando Python.

Depois, conhecemos duas ferramentas que permitem fazer esse trabalho de forma facilitada, o `uv` e o `poetry`. Vimos como funcionam com alguns exemplos, entendemos as vantagens de cada uma delas e no final falamos sobre quando usar cada uma dessas ferramentas.

REFERÊNCIAS

MAHAMULKAR, P. **Machine Learning Operations (MLOps) For Beginners**. 2024. Disponível em: <https://towardsdatascience.com/machine-learning-operations-mlops-for-beginners-a5686bfe02b2/>. Acesso em: 02 abril 2026.

MLOPS. **Machine Learning Operations**. 2026. Disponível em: <https://ml-ops.org/>. Acesso em: 21 abr. 2026. 02 abril 2026.

POTE, S. **Machine Learning in Production**. [s.l.]: BPB Publications, 2023.

PYTHON PACKAGING USER GUIDE. **Escrevendo seu pyproject.toml**. 2026. Disponível em: <https://packaging.python.org/pt-br/latest/guides/writing-pyproject-toml/>. Acesso em: 22 abr. 2026.

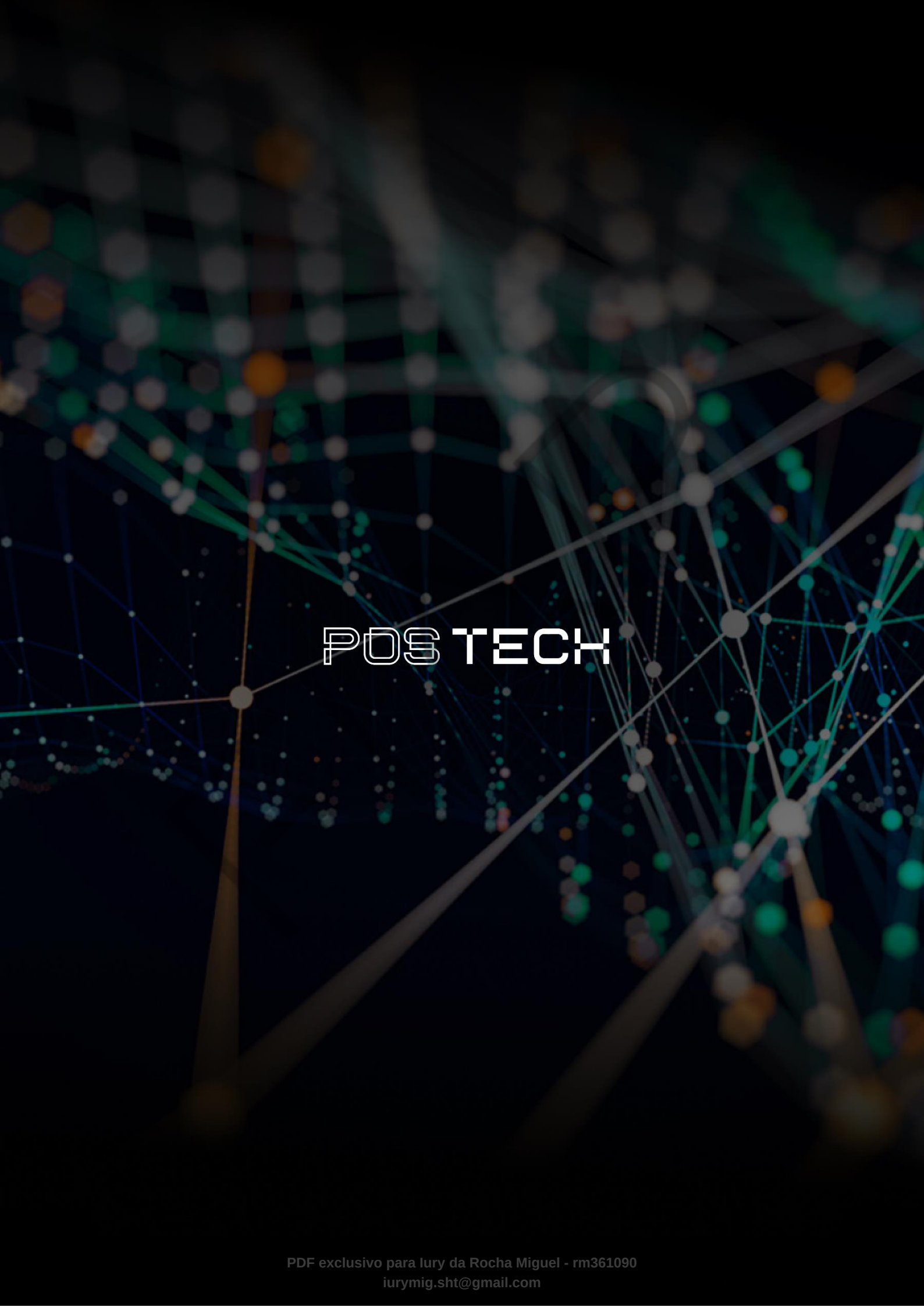
PYTHON-POETRY.ORG. **Poetry - Python dependency management and packaging made easy**. 2026. Disponível em: <https://python-poetry.org/docs/>. Acesso em: 22 abr. 2026.

UV. **uv**. 2026. Disponível em: <https://docs.astral.sh/uv/>. Acesso em: 22 abr. 2026.

PALAVRAS-CHAVE

Gerência de dependências. Lockfile. Ambientes virtuais.
uv vs Poetry.

EMANIP



POSTECH